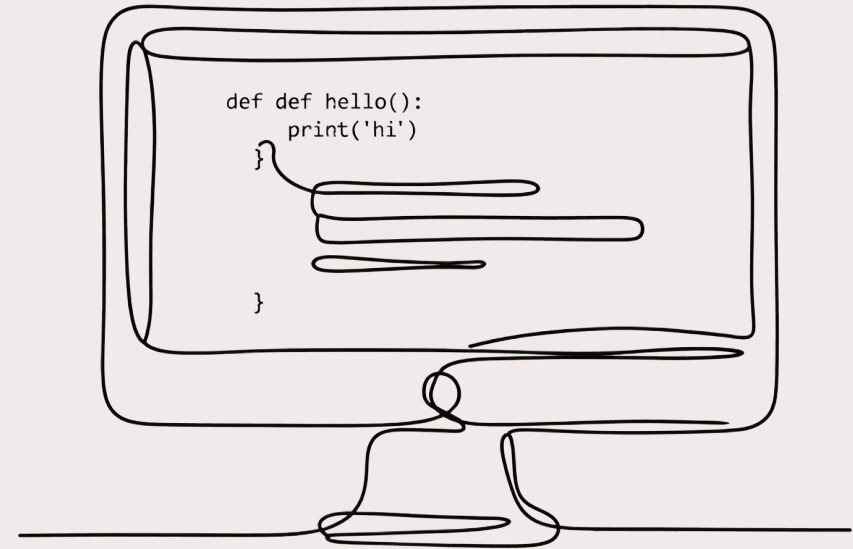


Control de Flujo, Estructuras de Datos y Módulos en Python

Una guía práctica y didáctica para dominar los fundamentos esenciales de Python: desde cómo dirigir el flujo de tu programa, hasta cómo organizar datos y reutilizar código con módulos.

PYTHON · PRINCIPIANTES



¿Qué es el Control de Flujo?

Las herramientas de control de flujo determinan el **orden en que se ejecutan las instrucciones** dentro de un programa. Sin ellas, el código simplemente correría de arriba hacia abajo, sin tomar decisiones ni repetir tareas. Con ellas, puedes construir programas inteligentes y dinámicos.

Decisiones

Ejecutar código solo cuando se cumple una condición específica.

Repetición

Repetir un bloque de código varias veces con bucles.

Control preciso

Interrumpir, saltar o continuar la ejecución según la lógica del programa.

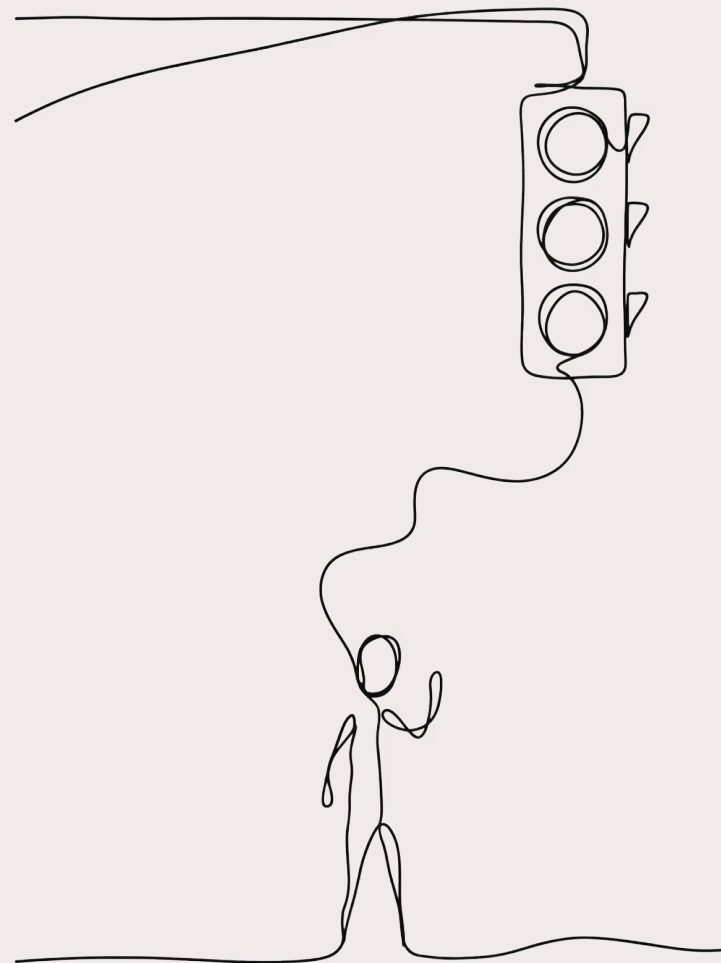
Sentencia if — Tomando Decisiones

La sentencia `if` permite ejecutar un bloque de código **únicamente cuando se cumple una condición**. Si la condición es verdadera (`True`), el bloque se ejecuta; si es falsa (`False`), se omite. Puedes añadir `elif` para más condiciones y `else` como caso por defecto.

```
edad = 18

if edad >= 18:
    print("Es mayor de edad")
elif edad >= 13:
    print("Es adolescente")
else:
    print("Es menor de edad")
```

- ❏ La indentación (sangría) es fundamental en Python. Todo el código dentro del `if` debe estar desplazado 4 espacios hacia la derecha.



Bucle for y la Función range()

El bucle for recorre los elementos de una secuencia (lista, cadena, tupla, etc.) uno por uno. La función range() genera una secuencia de números y es la pareja perfecta del for cuando necesitas repetir algo un número determinado de veces.

Bucle for con lista

```
for numero in [1, 2, 3]:  
    print(numero)  
# Salida: 1, 2, 3
```

Bucle for con range()

```
for i in range(5):  
    print(i)  
# Salida: 0, 1, 2, 3, 4
```

range(5) genera números del 0 al 4. Recuerda: el límite superior **no se incluye**.

Control del Bucle: break, continue y else

Estas tres sentencias te dan **control preciso** sobre la ejecución de un bucle. Permiten interrumpirlo, saltar una iteración o ejecutar código cuando termina normalmente.

break — Interrumpir el ciclo

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)  
# Imprime: 0, 1, 2, 3, 4
```

continue — Saltar una iteración

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)  
# Imprime: 0, 1, 3, 4
```

else — Fin normal del ciclo

```
for i in range(3):  
    print(i)  
else:  
    print("Fin del ciclo")  
# El else NO corre si hubo break
```

Argumentos en Funciones: Valores por Omisión, Palabras Clave y Listas Variables

Python ofrece formas muy flexibles de definir y llamar funciones. Puedes establecer **valores predeterminados**, usar **nombres de parámetros** al llamar una función, o incluso recibir una **cantidad variable** de argumentos.

1

Valores por omisión

```
def saludar(nombre="Usuario"):
    print("Hola", nombre)
```

```
saludar()      # Hola Usuario
saludar("Ana") # Hola Ana
```

2

Palabras clave

```
def persona(nombre, edad):
    print(nombre, edad)
```

```
persona(edad=20, nombre="Ana")
# El orden no importa
```

3

Listas de argumentos *args

```
def suma(*numeros):
    return sum(numeros)
```

```
print(suma(1, 2, 3, 4))
# Salida: 10
```

Sentencia `pass` y Desempaquetado

`pass` — Marcador de posición

No realiza ninguna acción. Es útil cuando Python requiere una instrucción sintácticamente válida pero tú aún no has escrito el código.

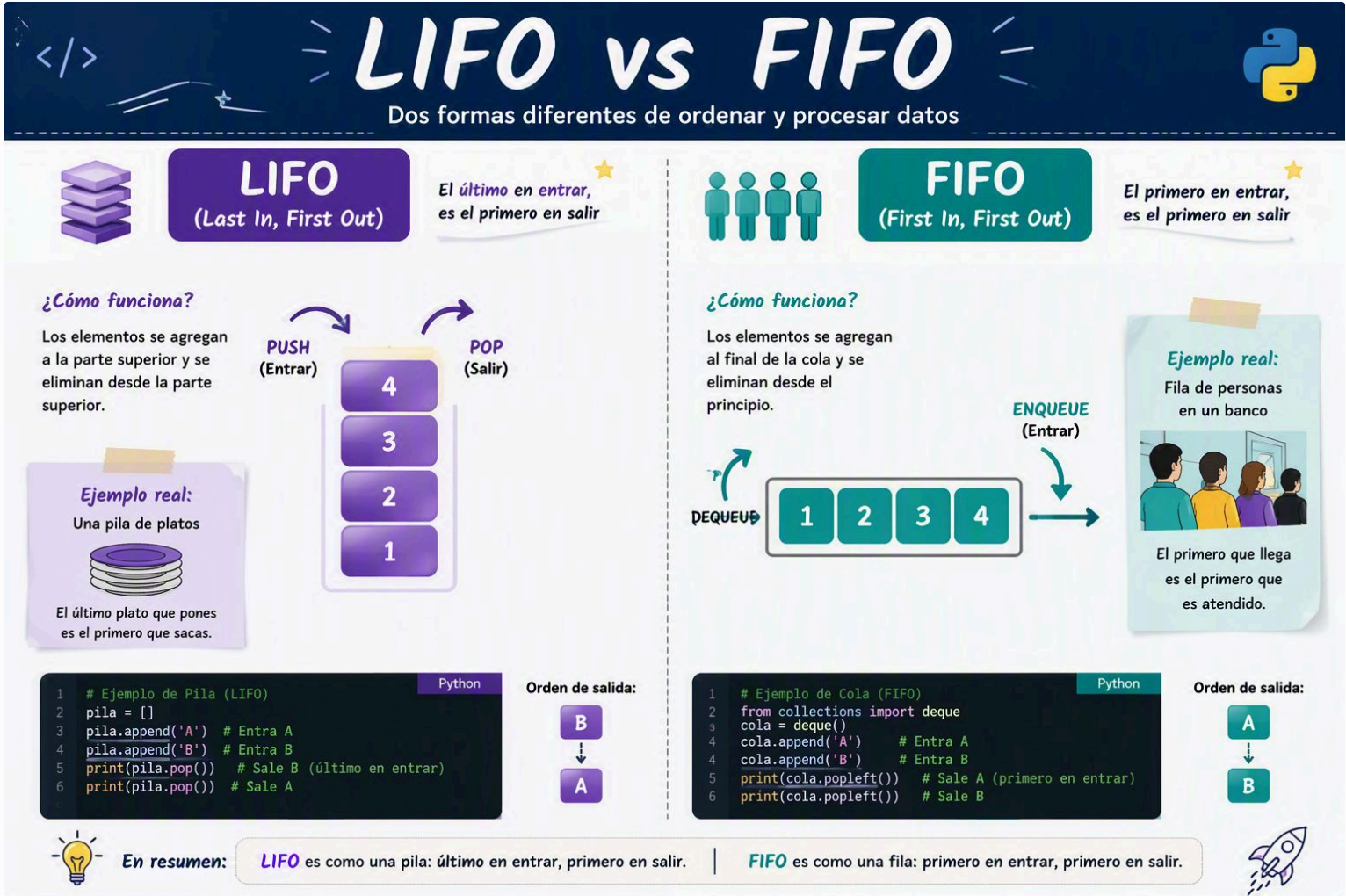
```
for i in range(5):  
    pass # TODO: agregar lógica después  
  
class MiClase:  
    pass # Clase vacía por ahora
```

Desempaquetado de secuencias

Permite asignar los elementos de una secuencia a variables individuales en una sola línea, de forma elegante y clara.

```
datos = ("Ana", 20)  
  
nombre, edad = datos  
  
print(nombre) # Ana  
print(edad) # 20
```

Estructuras de Datos Esenciales



Las estructuras de datos te permiten **organizar y almacenar información de manera eficiente**. Python incluye varias de ellas de forma nativa, cada una con sus propias reglas y casos de uso ideales.



Pilas (LIFO)

Último en entrar, primero en salir. Como una pila de platos: solo puedes agregar o quitar desde arriba.

¿Cuándo usarla? Historial de acciones (deshacer/rehacer), navegación de páginas, evaluación de expresiones matemáticas.

Métodos principales:

- `append(x)` — Agrega un elemento al tope
- `pop()` — Elimina y retorna el elemento del tope
- `[-1]` — Consulta el tope sin eliminarlo

```
pila = []
pila.append("página1")
pila.append("página2")
pila.append("página3")

print(pila[-1]) # "página3" (tope)
pila.pop()      # Retorna "página3"
pila.pop()      # Retorna "página2"
print(pila)     # ["página1"]
```



Listas Anidadas

Listas dentro de listas que forman estructuras multidimensionales.
Ideales para representar **matrices, tableros o tablas de datos**.

¿Cuándo usarlas? Tableros de juegos (ajedrez, tic-tac-toe), imágenes en píxeles, hojas de cálculo, grafos.

Acceso y recorrido:

- `lista[fila][columna]` — Acceso a un elemento
- Bucles anidados para recorrer toda la estructura
- Comprensión de listas para crear matrices rápidamente

```
# Tablero de tic-tac-toe

tablero = [
    ["X", "O", "X"],
    [" ", "X", " "],
    ["O", " ", "O"]
]

# Acceder a un elemento
print(tablero[1][1]) # "X" (centro)

# Recorrer toda la matriz
for fila in tablero:
    print(" | ".join(fila))
```



Colas (FIFO)

Primero en entrar, primero en salir. Como una fila de personas: el primero que llega es el primero en ser atendido.

¿Cuándo usarla? Procesamiento de tareas en orden, sistemas de mensajería, gestión de turnos o impresoras.

Métodos principales:

- `append(x)` — Agrega al final de la cola
- `popleft()` — Elimina y retorna el primero
- `appendleft(x)` — Agrega al inicio (deque bidireccional)

```
from collections import deque

cola = deque()
cola.append("tarea1")
cola.append("tarea2")
cola.append("tarea3")

print(cola)          # deque(['tarea1', 'tarea2', 'tarea3'])
procesado = cola.popleft() # "tarea1"
print(procesado)     # tarea1
print(len(cola))     # 2
```



Tuplas

Colecciones ordenadas e **inmutables**. Una vez creadas, sus elementos no pueden modificarse, lo que las hace más seguras y eficientes que las listas.

¿Cuándo usarlas? Coordenadas geográficas, configuraciones fijas, retorno de múltiples valores desde funciones, claves de diccionarios.

Métodos y características:

- `count(x)` – Cuenta cuántas veces aparece un valor
- `index(x)` – Devuelve el índice de un valor
- Soportan desempaquetado: `x, y = tupla`

```
# Coordenadas inmutables
ubicacion = (40.7128, -74.0060) # Nueva York
lat, lon = ubicacion
print(f"Latitud: {lat}, Longitud: {lon}")

# Retorno múltiple desde función
def estadisticas(datos):
    return min(datos), max(datos), sum(datos)/len(datos)

minimo, maximo, promedio = estadisticas([3, 7, 1, 9, 4])
print(f"Min: {minimo}, Max: {maximo}, Prom: {promedio}")
```


Técnicas de Iteración: `enumerate()` y `zip()`

Estas funciones integradas hacen que la iteración sea más expresiva y elegante, eliminando la necesidad de manejar índices manualmente o de iterar secuencias por separado.

`enumerate()` — Índice + Valor

Recorre una secuencia obteniendo el **índice y el valor** de cada elemento al mismo tiempo.

```
frutas = ["manzana", "pera", "uva"]

for indice, valor in enumerate(frutas):
    print(indice, valor)

# 0 manzana
# 1 pera
# 2 uva
```

`zip()` — Varias secuencias a la vez

Combina dos o más secuencias y las recorre en paralelo, emparejando elementos por posición.

```
nombres = ["Ana", "Luis"]
edades = [20, 25]

for nombre, edad in zip(nombres, edades):
    print(nombre, edad)

# Ana 20
# Luis 25
```

Módulos y Paquetes: Organiza y Reutiliza tu Código

Un **módulo** es un archivo `.py` con funciones, variables o clases reutilizables. Un **paquete** es una carpeta que agrupa varios módulos relacionados. Ambos son la base de un código organizado y profesional.

Crear y usar un módulo

```
# archivo: operaciones.py
def suma(a, b):
    return a + b

# En tu programa principal:
import operaciones
print(operaciones.suma(5, 3)) # 8
```

Explorar un módulo con `dir()`

```
import math
print(dir(math))
# Lista todas las funciones de math
```

Estructura de un paquete

```
mi_paquete/
|
├── __init__.py
├── modulo1.py
└── modulo2.py
```

Importar desde un paquete

```
from mi_paquete import modulo1

# o también:
import mi_paquete.modulo1

# Llamar una función:
mi_paquete.modulo1.mi_funcion()
```

El archivo `__init__.py` le indica a Python que esa carpeta es un paquete.

Glosario de Términos Clave

Aquí tienes un resumen de los conceptos fundamentales de Python que hemos explorado en esta presentación, organizados para una referencia rápida.

Término	Definición
Control de flujo	Mecanismo que determina el orden en que se ejecutan las instrucciones de un programa.
if / elif / else	Sentencias condicionales que ejecutan bloques de código según si una condición es verdadera o falsa.
Bucle for	Estructura que itera sobre los elementos de una secuencia (lista, rango, cadena, etc.).
range()	Función que genera una secuencia de números enteros. El límite superior no se incluye.
break	Sentencia que interrumpe y termina un bucle de forma inmediata.
continue	Sentencia que salta la iteración actual y pasa a la siguiente.
else (en bucle)	Bloque que se ejecuta cuando un bucle termina normalmente, sin haber sido interrumpido por break.
pass	Sentencia vacía usada como marcador de posición cuando se requiere una instrucción sintácticamente válida.
Desempaquetado	Asignación de los elementos de una secuencia a variables individuales en una sola línea.
Pila (Stack)	Estructura de datos LIFO (último en entrar, primero en salir).
Cola (Queue)	Estructura de datos FIFO (primero en entrar, primero en salir).
Tupla	Colección ordenada e inmutable de elementos.
Lista anidada	Lista que contiene otras listas como elementos; útil para representar matrices.
enumerate()	Función que recorre una secuencia devolviendo el índice y el valor de cada elemento.
zip()	Función que combina dos o más secuencias y las recorre en paralelo.
Módulo	Archivo .py que contiene funciones, variables o clases reutilizables.
Paquete	Carpeta que agrupa varios módulos relacionados, identificada por un archivo __init__.py.
dir()	Función que lista todos los atributos y métodos disponibles en un módulo u objeto.
*args	Parámetro especial que permite recibir una cantidad variable de argumentos posicionales en una función.
Indentación	Sangría obligatoria en Python que define los bloques de código.